VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

* * * * **

Machine Learning (C03117) - Semester 252

Assignment Report:

Cat vs. Dog Image Classification: A Comparative Study of Traditional and Deep Learning Pipelines

---

Instructor: Trương Vĩnh Lân

Class: CC01

Students: Nguyễn Mạnh Quốc Khánh - 2352525 **(Leader)**

Phan Ngọc Lan Chi - 2352137

Ngô Diễm Quyên - 2353031

Trần Lâm Anh - 2352067

Vũ Đức Việt Anh - 2352074

Ho Chi Minh City, May 2026

Abstract

This paper presents a complete image classification pipeline for binary cat–dog recognition, comparing a hybrid classical machine learning approach against an end-to-end deep learning extension. Beyond maximizing accuracy, the study systematically investigates how preprocessing strategy, feature extraction backbone, classifier selection, and reproducibility jointly influence practical modelling decisions.

The hybrid classical pipeline combines frozen pretrained CNN feature extraction with traditional classifiers. An empirical cleaning-threshold experiment confirms that a keep-all policy (10,028 images) is sufficient. An exhaustive grid search over 132 configurations — spanning four preprocessing strategies, three CNN backbones, and eleven classifier variants — identifies augmented preprocessing with EfficientNet-B0 features as the strongest representation. Although Soft Voting and Logistic Regression achieve equivalent top validation scores (≈99.70%), Logistic Regression is selected as the operational model due to substantially lower training cost and superior reproducibility. On a stratified held-out test set of 1,003 images, the model achieves 99.40% accuracy and 99.40% macro F1-score.

The deep learning extension fine-tunes EfficientNet-B0 via a two-phase transfer learning strategy, achieving 98.70% accuracy, 98.70% macro F1-score, and ROC-AUC of 0.9992. As both pipelines differ slightly in test partition due to an additional outlier handling step, comparisons are interpreted as indicative rather than split-identical.

Overall, results demonstrate that pretrained visual features are highly discriminative for this task, and that a classical classifier on frozen CNN features can match or exceed a fine-tuned neural network in both accuracy and deployability. These findings highlight the practical importance of distinguishing between the best validation candidate and the most operationally reproducible model.

Keywords: Cat–dog classification, image classification, hybrid classical pipeline, pretrained feature extraction, transfer learning, EfficientNet-B0, Logistic Regression, Soft Voting, model comparison.

Contents

List of Figures

List of Tables

Member list & Workload

No.	Full name	Student ID	Contribution	% done
1	Nguyễn Mạnh Quốc Khánh	2352525	Group leader; coordinated project direction and task allocation; designed the configuration-driven workflow; implemented and reviewed the hybrid classical pipeline; supervised feature extraction, Logistic Regression tuning, grid-search organization, result verification, and final report integration.	100%
2	Phan Ngọc Lan Chi	2352137	Prepared dataset description and exploratory data analysis; generated and interpreted class distribution, sample visualization, image-size distribution, and RGB colour-channel analysis; contributed to data preparation writing, preprocessing explanation, and report polishing.	100%
3	Ngô Diễm Quyên	2353031	Implemented and analyzed the image quality and cleaning-threshold experiment; summarized the minimal-safety keep-all policy; supported validation metric interpretation, confusion-matrix discussion, and misclassified-sample analysis.	100%
4	Trần Lâm Anh	2352067	Developed the end-to-end deep learning extension using EfficientNet-B0 transfer learning and fine-tuning; documented the augmentation strategy, training configuration, training dynamics, final deep learning metrics, and limitations of the deep learning run.	100%
5	Vũ Đức Việt Anh	2352074	Executed and consolidated classical grid-search experiments; organized saved feature files, model artifacts, and result tables; supported reproducibility checks, project folder structure, README/submission preparation, and final LaTeX formatting.	100%

Table 1: Member list & workload

1 Introduction

1.1 Background and Problem Statement

Image classification is a fundamental task in **machine learning** and **computer vision**. Its objective is to assign a semantic label to an input image based on visual content. This task appears in many practical applications, including object recognition, visual search, automated content organization, surveillance systems, medical image analysis, and autonomous technologies. Although modern deep learning models have achieved strong results in image recognition, building a reliable classification system still requires careful decisions about data quality, preprocessing, feature representation, model selection, computational cost, and evaluation methodology.

This project focuses on the binary classification of *Cat* and *Dog* images. While cat–dog classification is a common benchmark problem, it remains useful for evaluating a complete image classification workflow because the images vary substantially in breed, pose, scale, lighting condition, camera angle, background complexity, and visual quality. These variations make the task more meaningful than a simple two-class exercise and provide a suitable setting for comparing different modelling strategies.

The central problem of this project is to develop a **reliable**, **reproducible**, and **computationally practical** system that can classify an unseen image as either *Cat* or *Dog*. Rather than focusing only on the highest possible score, the project investigates how different design choices affect the final pipeline. These choices include cleaning policy, image transformation, pretrained backbone, classifier type, model-selection criterion, and training strategy.

Following the main requirement of the assignment, the primary approach is a **hybrid classical machine learning pipeline**. In this design, pretrained convolutional neural networks are used as frozen feature extractors, while traditional machine learning algorithms perform the final classification step. This allows the project to satisfy the traditional machine learning pipeline requirement while still using modern deep visual features as the representation stage.

In addition, an **end-to-end deep learning pipeline** is implemented as an extension. This pipeline fine-tunes EfficientNet-B0 directly on the cat–dog dataset using transfer learning. It is included to provide a broader comparison between frozen pretrained feature extraction and full neural network fine-tuning. However, because the deep learning pipeline applies an additional spatial outlier handling step, its evaluation split is not strictly identical to the hybrid classical pipeline. The comparison is therefore interpreted with this protocol difference in mind.

Overall, the project studies the balance between predictive performance, runtime cost, implementation complexity, and reproducibility. This perspective is important because the most complex model is not always the most suitable final choice. In the exhaustive classical experiment, Soft Voting is the top-ranked validation candidate, while Logistic Regression achieves the same rounded validation score with much lower training and tuning cost. For this reason, Logistic Regression is selected as the main operational model, and Soft Voting is reported as the strongest ensemble candidate from the validation search.

1.2 Project Objectives

The main objective of this project is to design, implement, evaluate, and compare image classification pipelines for the cat–dog classification task. The project covers the full machine learning workflow, from data exploration and preparation to feature extraction, model training, evaluation, and comparative analysis.

The specific objectives are grouped as follows:

- **Dataset understanding and preparation:** conduct exploratory data analysis to examine class distribution, sample images, raw image dimensions, RGB colour-channel characteristics, and potential quality issues; evaluate cleaning-threshold policies and select a final data preparation strategy based on empirical evidence.
- **Hybrid classical pipeline development:** build a traditional machine learning pipeline in which images are preprocessed, transformed into fixed-dimensional pre-trained CNN feature vectors, saved as NumPy feature files, and classified using traditional models such as Logistic Regression, linear Support Vector Machine, Random Forest, Soft Voting, and Stacking.
- **Configuration-based experimentation:** evaluate multiple preprocessing strategies, pretrained backbones, and classifier variants through an exhaustive classical experiment over 132 configurations, while keeping the pipeline flexible enough to change image size, feature extractor, and classifier settings.
- **Model selection and final evaluation:** select the main operational model using both validation performance and practical criteria such as training cost, tuning time, implementation simplicity, and reproducibility; evaluate the selected model on a held-out test set using accuracy, precision, recall, macro F1-score, confusion matrix, and misclassified examples.
- **Deep learning extension and comparison:** implement an end-to-end EfficientNet-B0 transfer learning pipeline as an extension, compare it with the hybrid classical approach, and discuss the trade-offs between frozen feature extraction and full fine-tuning.
- **Reproducible submission:** organize the implementation into a clear project structure with notebooks, modules, reports, saved feature files, and supporting documentation so that the notebook can be rerun and the experimental process can be verified.

1.3 Report Structure

The remainder of this report is organized as follows:

- **Section 2 – Dataset and Exploratory Data Analysis:** describes the dataset source, class distribution, representative images, image-size characteristics, RGB colour-channel properties, and the main implications of the exploratory analysis.
- **Section 3 – Data Preparation:** presents the supported image quality checks, cleaning-threshold experiment, final minimal-safety keep-all policy, image transformation steps, augmentation strategy, and stratified train-validation-test split.

- **Section 4 – System Design and Methodology:** explains the overall modelling design, including the hybrid classical feature-extraction pipeline, pretrained backbones, traditional classifiers, feature storage strategy, evaluation metrics, and the end-to-end deep learning extension.
- **Section 5 – Experimental Design:** describes the cleaning-threshold experiment, the 132-configuration classical grid-search protocol, validation and model-selection strategy, main operational configuration, and computational strategy.
- **Section 6 – Results: Hybrid Classical Pipeline:** reports the classical grid-search results, analyzes the effects of backbone, preprocessing, and classifier choice, explains the selection of Logistic Regression as the main operational model, and presents final test performance, class-level metrics, confusion matrix, and misclassified samples.
- **Section 7 – Results: End-to-End Deep Learning Extension:** presents the EfficientNet-B0 transfer learning architecture, augmentation strategy, training configuration, training dynamics, final test performance, and discussion of the completed deep learning run.
- **Section 8 – Comparative Analysis:** compares the hybrid classical pipeline and the deep learning extension in terms of performance, computational cost, reproducibility, implementation complexity, and practical suitability.
- **Section 9 – Limitations and Future Work:** discusses the main limitations of the current study and proposes future improvements, including external validation, stricter duplicate handling, repeated deep learning runs, shared fixed splits, additional backbones, and interpretability analysis.

2 Dataset and Exploratory Data Analysis

2.1 Dataset Source and Description

This project uses the public *Cat* and *Dog* dataset from Kaggle for a supervised **binary image classification** task. The dataset consists of real-world images from two target classes: *Cat* and *Dog*. Each image is associated with one class label, and the objective is to predict whether an unseen image contains a cat or a dog.

The raw dataset contains **10,028 images**, including **5,011 cat images** and **5,017 dog images**. Since the two classes are almost equally represented, the dataset provides a suitable basis for training and evaluating binary classification models without severe class imbalance. This is important because a strongly imbalanced dataset may cause a model to favor the majority class and produce misleadingly high accuracy.

Although cat–dog classification is a common benchmark task, the dataset still presents realistic visual challenges. Images vary in breed, pose, scale, background, lighting condition, camera angle, resolution, and image quality. Some images contain clearly visible animals, while others include cluttered backgrounds, partial occlusion, low resolution, or poor focus. These variations make the dataset appropriate for evaluating the robustness of both the **hybrid classical feature-extraction pipeline** and the **end-to-end deep learning extension**.

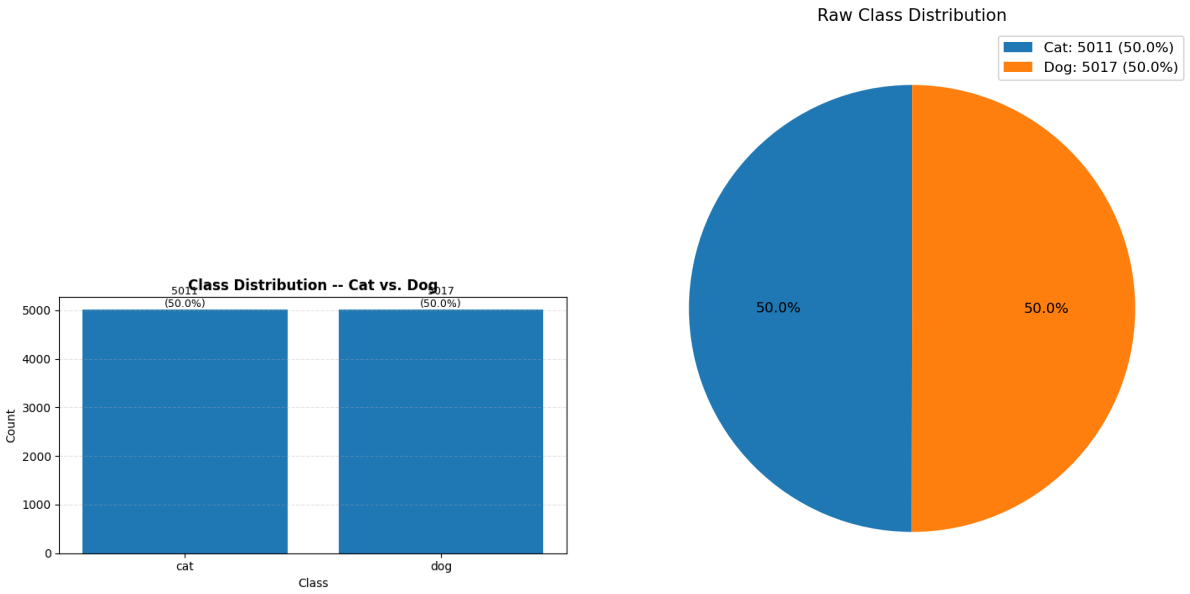
Table 2: Overview of the raw dataset

Dataset Stage	Total Images	Cat Images	Dog Images
Raw dataset	10,028	5,011	5,017

Exploratory Data Analysis (EDA) was conducted before data preparation and model training to understand the structure, visual characteristics, and possible quality issues of the dataset. For image data, the analysis focuses on class distribution, representative samples, raw image dimensions, RGB colour-channel properties, and data quality observations. These findings guide later design choices such as RGB conversion, resizing, ImageNet normalization, augmentation, stratified splitting, and cleaning-policy selection.

2.2 Class Distribution

Class distribution analysis was performed to examine whether the raw dataset is balanced between the *Cat* and *Dog* classes. As shown in Figure 1, the dataset contains **5,011 cat images** and **5,017 dog images**. The difference between the two classes is only six images, indicating that the dataset is nearly perfectly balanced.



(a) Bar chart (b) Pie chart

Figure 1: Class distribution of the raw dataset. Both visualizations show that the dataset is nearly balanced, with 5,011 Cat images and 5,017 Dog images.

This balanced distribution is beneficial for both training and evaluation. During training, the two classes contribute almost equally to the learning process, reducing the risk that the model becomes biased toward one class. During evaluation, accuracy becomes more meaningful because the test performance is not dominated by a majority class.

However, accuracy alone is not sufficient to fully evaluate classification performance. A model may achieve high accuracy while still making uneven errors across classes. Therefore, this project also uses **precision**, **recall**, **macro F1-score**, and **confusion matrix analysis** to examine whether the models perform consistently on both cat and dog images.

2.3 Sample Image Visualization

Representative sample images were visualized to gain an initial understanding of the dataset content. Figure 2 presents a grid of sample images selected from the raw dataset. The first row contains cat images, while the second row contains dog images.

Figure 2: Representative sample images from the raw dataset. The first row contains cat images, while the second row contains dog images.

The sample images show that the dataset contains considerable **visual diversity**. The animals appear in different poses, scales, positions, and environments. Some images contain close-up views of the animal, while others show the animal from a distance or against a complex background. Lighting conditions also vary across images, ranging from bright outdoor scenes to darker indoor settings.

This visual variation has important implications for model design. Since the classification task cannot rely on a single fixed visual pattern, the model must extract **robust visual representations** that generalize across different image conditions. In the hybrid classical pipeline, this motivates the use of pretrained convolutional neural network backbones as fixed feature extractors. In the deep learning extension, it motivates the use of transfer learning and data augmentation.

The sample visualization also suggests that some images may contain quality-related issues such as blur, low resolution, partial occlusion, or visually uninformative content. These observations support the need for systematic image quality analysis. However, difficult images are not automatically removed, because they may still represent realistic visual variation that the model should learn to handle.

2.4 Image Size Distribution

The raw images in the dataset do not share a fixed resolution. Therefore, the width and height of all images were analyzed before applying transformations. Figure 3 visualizes the distribution of raw image dimensions using width and height as two axes.

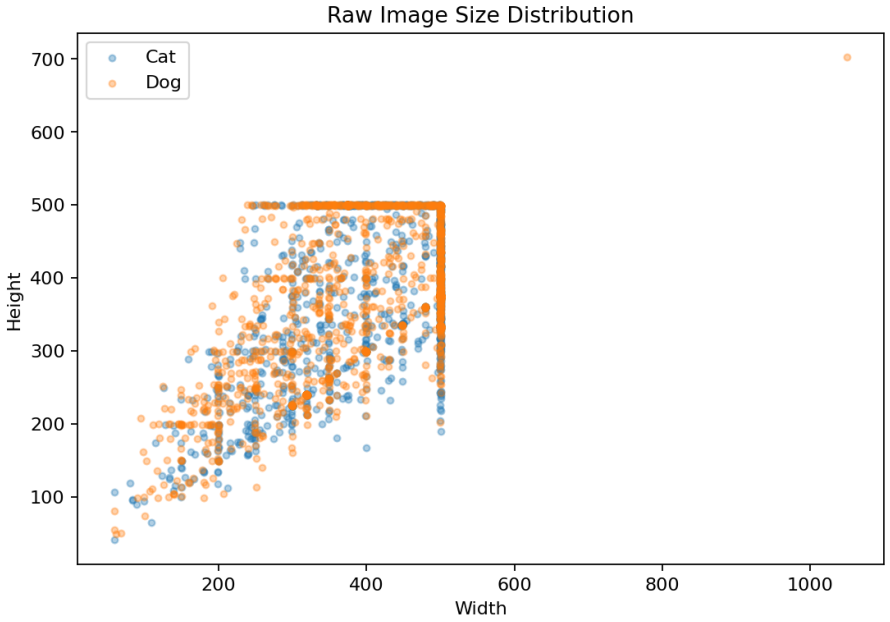


Figure 3: Raw image size distribution represented by image width and height.

The figure shows that the dataset contains images with **heterogeneous dimensions**. Some images are relatively small, while others are much larger. This variation is common in real-world image datasets, but it creates a technical requirement for preprocessing because pretrained convolutional neural network backbones require inputs of a consistent size.

In this project, images are transformed into a fixed input size of 224×224 **pixels** for the hybrid classical pipeline. This size is compatible with the pretrained backbones used for feature extraction, including *ResNet18*, *VGG16*, and *EfficientNet-B0*. It is also suitable for mini-batch processing, where image tensors must have consistent spatial dimensions.

The image size distribution also motivates the evaluation of **minimum-resolution filtering**. Extremely small images may lose important semantic details after resizing and may produce weak feature representations. Therefore, undersized-image checks are supported by the data preparation pipeline. Nevertheless, the final reported cleaning policy is determined empirically through the cleaning-threshold experiment rather than by applying aggressive filtering by default.

2.5 RGB Colour Channel Analysis

Since the pretrained convolutional neural network backbones used in this project are trained on RGB images, it is important to examine the colour-channel characteristics of the dataset. Each image is converted to RGB format before being used for feature extraction or deep learning training. Figure 4 presents the distribution of mean red, green, and blue channel intensities for each class.

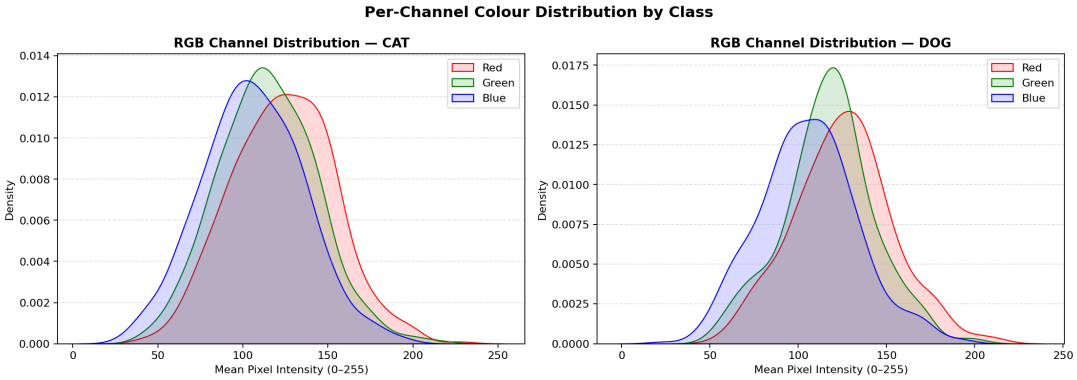


Figure 4: Per-channel RGB colour distribution by class based on mean pixel intensity.

The RGB channel distributions provide a general overview of the colour properties of cat and dog images. Although colour may contribute useful information, the distributions of the two classes overlap substantially. This indicates that **colour information alone is not sufficient** for reliable classification.

This finding supports the use of **deeper visual representations**. A reliable cat–dog classifier should capture not only colour patterns, but also object-level features such as shape, texture, edges, facial structure, fur patterns, and body characteristics. In the hybrid classical pipeline, these features are extracted using pretrained CNN backbones and then passed to traditional classifiers. In the deep learning extension, the model learns and adapts such representations directly through transfer learning.

The RGB analysis also supports the use of **ImageNet normalization**. Since the selected pretrained backbones were originally trained using ImageNet-style preprocessing, normalizing images with ImageNet mean and standard deviation values helps align the input data with the distribution expected by the pretrained models.

2.6 Summary and Pipeline Implications

Table 3 summarizes the main findings from the dataset exploration and their implications for the later stages of the project.

Table 3: Summary of EDA findings and pipeline implications

EDA Aspect	Main Observation	Pipeline Implication
Dataset size and labels	The raw dataset contains 10,028 labelled images from two classes.	The dataset is suitable for supervised binary classification.
Class distribution	Cat and Dog classes are nearly balanced.	Stratified splitting is appropriate; accuracy and macro F1-score are meaningful.
Sample visualization	Images vary in pose, scale, background, lighting condition, and visual quality.	Robust visual feature extraction is required for both the hybrid classical pipeline and the deep learning extension.
Image size distribution	Raw images have inconsistent dimensions.	Images must be resized or transformed to a fixed input size.
RGB channel distribution	Colour distributions overlap substantially between classes.	Colour alone is insufficient; pretrained deep visual representations are needed.
Data quality	Some images may be blurry, undersized, corrupted, duplicated, or visually uninformative.	Quality checks and threshold analysis are needed, but aggressive filtering should only be applied if it provides stable practical benefit.

Overall, the exploratory analysis shows that the dataset is appropriate for the **binary cat–dog classification task**. The near-balanced class distribution supports fair supervised learning, while the diversity in image appearance makes the task meaningful for evaluating model robustness. At the same time, the heterogeneous image sizes and potential quality issues confirm the need for careful **data preparation** and empirical cleaning-policy selection.

These findings provide the foundation for the next section, where the dataset preparation strategy is described in detail. In particular, the next section explains the supported image quality checks, the threshold experiment used to evaluate cleaning policies, the final decision to retain all 10,028 usable images, and the transformation steps used before feature extraction and model training.

3 Data Preparation

Data preparation is an essential stage in the image classification workflow because raw image datasets often contain inconsistent image sizes, varied file formats, unreadable samples, duplicated images, and visually weak inputs. The exploratory data analysis in Section 2 shows that the cat–dog dataset is nearly balanced, but the images differ substantially in size, background complexity, lighting condition, colour distribution, and visual quality. Therefore, the dataset must be standardized before feature extraction and model training.

In this project, data preparation is designed primarily for the **hybrid classical feature-extraction pipeline**. Standardized images are required before they can be passed through pretrained convolutional neural network backbones to produce fixed-dimensional feature vectors. The same original dataset is also used as the basis for the end-to-end deep learning extension, although the deep learning pipeline applies an additional spatial outlier handling step. Therefore, the comparison between the two pipelines should be interpreted as indicative rather than strictly split-identical.

3.1 Data Quality Issues and Cleaning Criteria

The first objective of data preparation is to identify potential quality-related issues in the raw dataset. Although most images are usable, some samples may negatively affect feature extraction or model evaluation. The preprocessing pipeline therefore supports optional checks for corrupted images, duplicated or near-duplicated images, undersized images, blurry images, near-monochrome images, over-dark or over-bright images, and low-information samples.

Corrupted or unreadable images cannot be loaded properly during preprocessing or model training. Duplicate or near-duplicate images may make evaluation results overly optimistic if visually similar samples appear across different data splits. Undersized images may contain insufficient semantic information and may lose further detail after resizing. Blurry images may reduce the visibility of important edges, textures, and object boundaries. Near-monochrome, extremely dark, extremely bright, or low-information images may provide weak visual signals for both feature extraction and model training.

However, difficult images should not be removed automatically. Some blurry, low-resolution, partially occluded, or visually unusual images may still represent realistic variation that a practical classifier should learn to handle. For this reason, the project treats cleaning as an empirical model-selection question rather than a purely rule-based filtering step.

The image cleaning criteria supported by the preprocessing pipeline are summarized in Table 4.

Table 4: Image cleaning criteria supported by the preprocessing pipeline

Criterion	Purpose	Action When Enabled
File readability	Detect images that cannot be opened or decoded correctly.	Flag or remove unreadable files.
Duplicate detection	Detect exact or near-duplicate images using perceptual hashing.	Flag or remove near-duplicates.
Minimum resolution	Identify images with insufficient width or height for reliable visual representation.	Flag or remove undersized images.
Blur detection	Detect images with weak sharpness, unclear edges, or poor focus.	Flag or remove severely blurry images.
Near-monochrome detection	Identify images with extremely limited colour and texture variation.	Flag or remove near-monochrome images.
Brightness and contrast checks	Identify images that are extremely dark, extremely bright, or visually low-information.	Flag visually weak images.
RGB compatibility	Ensure that valid images have three colour channels compatible with pretrained CNN backbones.	Convert valid images to RGB.

The preprocessing module also supports configurable threshold parameters, as shown in Table 5. These parameters make the cleaning stage flexible and allow different filtering policies to be tested empirically.

Table 5: Main cleaning threshold parameters

Cleaning Rule	Config Key	Value Used
Minimum image side length	`min_side`	64
Maximum aspect extremity	`max_aspect_extremity`	5.0
Minimum blur score	`min_blur_laplacian`	40.0
Minimum entropy	`min_entropy`	0.0
Maximum near-monochrome ratio	`max_near_mono_ratio`	0.92
Maximum dark pixel ratio	`max_dark_ratio`	0.98
Maximum bright pixel ratio	`max_bright_ratio`	0.98
Minimum mean saturation	`min_mean_sat`	0.0
Minimum chroma mean	`min_chroma_mean`	0.0
Minimum center saliency ratio	`min_center_saliency_ratio`	0.0
Maximum compression artifact score	`max_compression_artifact`	Disabled
Duplicate hash distance threshold	`duplicate_hamming_threshold`	4

Although the pipeline supports these checks, the final reported experiment does not apply aggressive image filtering. The final cleaning policy is selected through the threshold analysis described in the next subsection.

3.2 Cleaning Threshold Analysis and Final Policy

To avoid selecting cleaning thresholds purely by intuition, a separate **cleaning-threshold experiment** was conducted. The purpose of this experiment was to determine whether quality-based filtering provides a stable and practically meaningful benefit for classification performance.

In this analysis, EfficientNet-B0 was used as a proxy feature extractor for all **10,028 images**, producing a feature matrix of shape $(10028, 1280)$. A Logistic Regression classifier was then used as a proxy model to compare different cleaning presets. The keep-all baseline already achieved a strong proxy macro F1-score of approximately **0.9935**. Some filtering presets produced small improvements under a single random seed, but these gains were not stable across seeds and often introduced undesirable trade-offs such as lower dataset retention or shifted class balance.

Based on this analysis, no aggressive cleaning preset provided a reliable improvement while preserving retention and class balance. Therefore, the final selected policy is a **minimal-safety / keep-all policy**. Under this policy, all usable images are retained for the reported traditional pipeline experiments.

Table 6: Cleaning-threshold analysis summary

Component	Result
Dataset size	10,028 images
Proxy feature extractor	EfficientNet-B0
Proxy feature shape	$(10028, 1280)$
Proxy classifier	Logistic Regression
Keep-all baseline macro F1-score	Approximately 0.9935
Stability finding	No aggressive cleaning preset produced a stable practical gain while preserving retention and class balance.
Final cleaning policy	Minimal-safety / keep all usable images
Final retention	10,028 / 10,028 images retained
Removed images	0

This experiment is important because it provides empirical justification for retaining the full usable dataset. The project does not claim that blurry, undersized, near-monochrome, or duplicated images were removed in the final reported configuration. Instead, the cleaning module supports these checks, while the final experiments keep all 10,028 usable images because aggressive filtering does not provide a stable practical benefit.

3.3 Dataset Retention

Since the final selected cleaning policy retains all usable images, no samples are removed from the dataset in the reported traditional pipeline experiments. The final dataset there-

fore remains nearly balanced between the Cat and Dog classes.

Table 7: Dataset size before and after final cleaning policy

Dataset Stage	Total Images	Cat Images	Dog Images
Raw dataset	10,028	5,011	5,017
Final retained dataset	10,028	5,011	5,017
Removed images	0	0	0

Although no images are removed in the reported experiments, the preprocessing module remains reusable for future datasets where more aggressive cleaning may be necessary. This design allows the pipeline to flag corrupted, duplicated, blurry, low-information, or visually extreme samples without forcing image removal when the empirical evidence does not support it.

3.4 Image Transformation and Normalization

After the final retention policy is selected, all valid images are transformed into a standardized format. This step is necessary because pretrained CNN backbones require input images with consistent dimensions, channel format, and numerical scale.

Each image is first converted to RGB format to ensure compatibility with pretrained convolutional neural networks. Images are then resized to a fixed input size of 224×224 **pixels**, which is used throughout the hybrid classical feature-extraction pipeline. Next, images are converted into tensor representations and normalized using the standard ImageNet mean and standard deviation values. Since the pretrained backbones were originally trained using ImageNet-style preprocessing, this normalization helps align the input distribution with the expected pretrained feature space.

Separate transformation pipelines are used for training and evaluation. The training transform may include stochastic augmentation operations, while the validation and testing transforms remain deterministic to ensure stable evaluation results.

Table 8: Image transformation and normalization steps for the hybrid classical pipeline

Step	Description
RGB conversion	Converts all valid images into a three-channel RGB format.
Resizing	Standardizes each image to 224×224 pixels.
Tensor conversion	Converts images into numerical tensor format for pretrained backbone input.
ImageNet normalization	Normalizes images using the mean and standard deviation expected by ImageNet-pretrained CNN backbones.

3.5 Data Augmentation Strategy

Data augmentation is applied only to the training pipeline in order to increase visual diversity and improve model robustness. The validation and testing datasets are not augmented because they are used for stable and consistent performance evaluation.

For the **hybrid classical pipeline**, the selected augmented preprocessing strategy applies a lightweight set of transformations before frozen feature extraction. The training transform consists of resizing to 224×224, random horizontal flipping, colour jittering, tensor conversion, and ImageNet normalization. The evaluation transform consists only of resizing, tensor conversion, and ImageNet normalization.

Table 9: Classical pipeline transformation strategy

Transform	Purpose
Resize to 224×224	Standardizes image size for pretrained CNN feature extraction.
Random horizontal flip	Simulates natural left-right variation in animal orientation.
Colour jitter	Improves robustness to lighting, contrast, and colour variation.
Tensor conversion	Converts images into numerical tensor format.
ImageNet normalization	Aligns the input image distribution with the pretrained backbone's expected input distribution.

The end-to-end deep learning pipeline uses a stronger dynamic augmentation strategy during mini-batch training, including random resized cropping and random rotation. These deep learning-specific transformations are discussed separately in Section 7. Keeping the two augmentation descriptions separate avoids implying that both pipelines use exactly the same preprocessing protocol.

3.6 Train-Validation-Test Splitting

After preprocessing, the dataset for the hybrid classical pipeline is divided into training, validation, and testing subsets using a stratified **80/10/10 split**. Stratification preserves the Cat/Dog ratio across all subsets and ensures balanced representation during training and evaluation.

The training set is used to fit the classical classifiers. Hyperparameter tuning is performed using cross-validation within the training features, while the validation set is used to compare candidate preprocessing, backbone, and classifier configurations. The testing set is reserved strictly for final evaluation after the main operational model has been selected. This separation helps reduce data leakage and provides a more reliable estimate of model performance on unseen data.

Table 10: Dataset distribution after stratified splitting for the hybrid classical pipeline

Split	Total	Cat	Dog	Cat/Dog Ratio
Training	8,022	4,009	4,013	49.98% / 50.02%
Validation	1,003	501	502	49.95% / 50.05%
Testing	1,003	501	502	49.95% / 50.05%
Total	**10,028**	**5,011**	**5,017**	**49.97% / 50.03%**

The split distribution shows that class balance is preserved across the training, validation, and testing subsets. This makes accuracy more meaningful and supports the use of macro F1-score as a balanced evaluation metric. The end-to-end deep learning extension follows a similar evaluation protocol, but because it applies an additional spatial outlier handling step, its final test size differs slightly from the traditional pipeline.

3.7 Summary of Prepared Dataset

The data preparation stage converts the raw dataset into a standardized and experimentally reliable dataset for the hybrid classical feature-extraction pipeline. The key decision in this section is that the final reported configuration uses a **minimal-safety cleaning policy** and retains all **10,028** usable images. This decision is based on the cleaning-threshold experiment, which shows that aggressive filtering does not provide a stable practical improvement under the selected evaluation criteria.

For the main traditional pipeline, images are converted to RGB, resized to 224×224, transformed into tensors, normalized using ImageNet statistics, and split into stratified training, validation, and testing subsets. Training images may receive lightweight augmentation before frozen feature extraction, while validation and testing images use deterministic transforms.

The prepared images are then passed through pretrained CNN backbones to extract feature vectors for traditional classifiers. In the final main notebook, the selected operational configuration uses **augmented preprocessing**, **EfficientNet-B0** frozen feature extraction, and **Logistic Regression** classification. The next section describes this system design and methodology in detail.

4 System Design and Methodology

This section describes the system design and methodology used in the project. The study contains two modelling approaches. The first and primary approach is a **hybrid classical machine learning pipeline**, where pretrained convolutional neural networks are used as frozen feature extractors and traditional machine learning models are used as final classifiers. The second approach is an **end-to-end deep learning extension**, where EfficientNet-B0 is fine-tuned directly on the cat–dog image dataset.

The hybrid classical pipeline is the main required pipeline of the assignment because it follows the traditional machine learning workflow: exploratory data analysis, preprocessing, feature extraction, classifier training, and evaluation. The deep learning pipeline is implemented as an additional extension to compare frozen feature extraction with full neural network fine-tuning.

4.1 Overview of the Two Modelling Approaches

The two modelling approaches are designed to answer different experimental questions.

The **hybrid classical pipeline** asks whether pretrained visual representations are strong enough for a traditional classifier to separate Cat and Dog images effectively. In this approach, images are first standardized through preprocessing. They are then passed through pretrained CNN backbones such as ResNet18, VGG16, and EfficientNet-B0. The CNN backbones are used only as fixed feature extractors; their weights are not updated during classifier training. The extracted feature vectors are then used as input for traditional machine learning classifiers such as Logistic Regression, Linear SVM, Random Forest, Soft Voting, and Stacking.

The **end-to-end deep learning pipeline** asks whether fine-tuning a pretrained neural network can provide additional benefits over frozen feature extraction. In this approach, EfficientNet-B0 is adapted directly to the cat–dog classification task through transfer learning. The classification head is trained first while the backbone is frozen, and the full network is then fine-tuned with a smaller learning rate.

The two approaches are complementary. The hybrid classical pipeline emphasizes computational efficiency, reproducibility, and controlled comparison of classifiers. The deep learning extension emphasizes task-specific representation adaptation through neural network optimization.

4.2 Hybrid Classical Pipeline Architecture

4.2.1 Overall Pipeline Design

The hybrid classical pipeline combines modern pretrained feature extraction with traditional machine learning classification. Instead of training a CNN from scratch, the project uses pretrained CNN backbones to transform each image into a compact numerical feature vector. A traditional classifier is then trained on these extracted features.

The overall workflow can be summarized as follows:

$$\text{Image} \xrightarrow{\text{preprocessing}} \text{Preprocessed Image} \xrightarrow{\text{frozen CNN backbone}} \text{Feature Vector} \xrightarrow{\text{classical classifier}} \text{Prediction} \tag{1}$$

This structure separates the representation stage from the classification stage. Such separation has two main advantages. First, it makes the experiment computationally practical because extracted features can be saved and reused for multiple classifier experiments. Second, it allows different classifiers to be compared under the same feature representation, making the comparison more controlled.

The hybrid pipeline is also suitable for the assignment requirement because the final decision model is still a traditional machine learning classifier. The pretrained CNN is used as a feature extraction method rather than as an end-to-end classifier.

4.2.2 Pretrained Feature Extraction

Feature extraction is the central component of the hybrid classical pipeline. Raw pixel values are not used directly as classifier inputs because they are high-dimensional and sensitive to small changes in lighting, position, scale, and background. Instead, each image is passed through a pretrained convolutional neural network to obtain a higher-level visual representation.

Let x_i denote an input image and f_θ denote a pretrained CNN backbone with fixed parameters θ. The extracted feature vector z_i is defined as:

$$z_i = f_\theta(x_i), \tag{2}$$

where z_i is the learned representation used as input for the downstream traditional classifier.

Three pretrained backbones are considered in the classical experiments: **ResNet18**, **VGG16**, and **EfficientNet-B0**. These backbones are selected because they represent different convolutional network designs and computational characteristics. ResNet18 is a lightweight residual network, VGG16 is a deeper sequential convolutional baseline, and EfficientNet-B0 is an efficient architecture that balances depth, width, and resolution.

Table 11: Pretrained backbones considered for feature extraction

Backbone	**Role in the Experiment**
ResNet18	A lightweight residual CNN used as an efficient baseline for pretrained feature extraction.
VGG16	A deeper sequential convolutional network used as a classical CNN baseline with a simple architecture.
EfficientNet-B0	An efficient CNN architecture used as the main feature extractor because it provides the strongest representation in the classical experiments.

For the main operational pipeline, **EfficientNet-B0** is selected as the feature extractor. Its pretrained classification head is removed, and the backbone output is converted into a fixed-length feature vector. EfficientNet-B0 produces **1280-dimensional** feature vectors for each image.

Using a frozen pretrained backbone has practical benefits. It reduces training cost because the CNN parameters are not updated, and it improves reproducibility because feature extraction is deterministic once preprocessing and model weights are fixed. It also makes classifier comparison more reliable because all classifiers can be trained on the same saved feature vectors.

4.2.3 Traditional Machine Learning Classifiers

After feature extraction, traditional machine learning classifiers are trained on the extracted feature vectors. The project evaluates several classifier families and variants to examine whether linear models, tree-based methods, or ensemble methods work best on pretrained CNN features.

Logistic Regression is used as a simple linear classifier. It estimates class probabilities from the extracted feature vectors and is effective when the pretrained backbone already produces a feature space where Cat and Dog images are close to linearly separable.

Linear Support Vector Machine is used as a margin-based classifier for high-dimensional feature spaces. It provides a strong linear baseline and is useful for comparing margin-based decision boundaries with probabilistic Logistic Regression.

Random Forest is included as a non-linear tree-based ensemble method. It tests whether non-linear decision rules can improve performance on extracted CNN features, although it may require more computation and may not always outperform linear classifiers when the features are already highly discriminative.

Soft Voting combines predicted probabilities from multiple base classifiers. It is evaluated to determine whether combining several decision patterns can improve validation performance.

Stacking trains a meta-classifier on top of base classifier predictions. It is a stronger ensemble strategy, but it also requires higher training cost because multiple base models and an additional meta-model must be fitted.

Table 12: Traditional classifiers considered in the hybrid classical pipeline

Classifier	Purpose in the Pipeline
Logistic Regression	Tests whether pretrained CNN features are linearly separable; selected as the main operational classifier due to its strong performance and low cost.
Linear SVM	Provides a margin-based linear baseline for high-dimensional feature vectors.
Random Forest	Tests whether non-linear tree-based decision rules improve performance on extracted features.
Soft Voting	Combines predicted probabilities from multiple classifiers; reported as the top-ranked ensemble candidate in the validation search.
Stacking	Uses predictions from base classifiers as inputs to a meta-classifier; included as a stronger but more expensive ensemble method.

Although Soft Voting is the top-ranked validation candidate in the exhaustive grid-search experiment, Logistic Regression is selected as the main operational classifier. This is because Logistic Regression reaches the same rounded validation score as Soft Voting while requiring much lower training and tuning cost. Therefore, the final model choice is based on a practical model-selection criterion rather than validation rank alone.

4.2.4 Feature Storage Strategy

After feature extraction, the generated feature matrices and corresponding labels are stored as NumPy files. This step is important for both computational efficiency and reproducibility. Feature extraction through CNN backbones is more expensive than training most traditional classifiers, so saving features avoids repeated CNN forward passes when comparing multiple classifiers.

For the main EfficientNet-B0 feature set, the extracted feature shapes are:

- `X_train`: $(8022, 1280)$;
- `X_val`: $(1003, 1280)$;
- `X_test`: $(1003, 1280)$.

For each data split, the saved files include:

- `X_train.npy`: extracted feature vectors for the training set;
- `y_train.npy`: labels for the training set;
- `X_val.npy`: extracted feature vectors for the validation set;
- `y_val.npy`: labels for the validation set;
- `X_test.npy`: extracted feature vectors for the testing set;
- `y_test.npy`: labels for the testing set.

This storage strategy separates feature extraction from classifier training. As a result, different classifiers can be trained, tuned, and evaluated using the same extracted feature representation. This makes the comparison fairer and makes the notebook faster to rerun.

4.2.5 Evaluation Metrics

The hybrid classical pipeline is evaluated using standard classification metrics: accuracy, precision, recall, F1-score, macro F1-score, and confusion matrix analysis. These metrics provide complementary views of model performance.

Accuracy measures the overall proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{3}$$

Precision measures the proportion of predicted positive samples that are actually positive:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{4}$$

Recall measures the proportion of actual positive samples that are correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{5}$$

The F1-score is the harmonic mean of precision and recall:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{6}$$

For binary classification, these metrics are computed for both Cat and Dog classes. Macro F1-score is calculated by averaging the F1-scores of the two classes equally. This is useful because it evaluates whether the classifier performs consistently across both categories rather than only maximizing overall accuracy.

The confusion matrix is also used to examine the types of classification errors made by the model. It shows how many Cat images are predicted as Cat or Dog, and how many Dog images are predicted as Cat or Dog. This provides a more detailed view of model behavior than a single aggregate metric.

4.3 End-to-End Deep Learning Pipeline Architecture

4.3.1 Model Architecture and Transfer Learning Strategy

The end-to-end deep learning extension uses **EfficientNet-B0** as the pretrained backbone. This choice is consistent with the classical experiments, where EfficientNet-B0 provides the strongest extracted feature representation among the tested backbones.

Unlike the hybrid classical pipeline, where the backbone is frozen and used only to generate feature vectors, the deep learning pipeline adapts the model through transfer learning and fine-tuning. The original classification head of EfficientNet-B0 is replaced with a task-specific classification head for two output classes: Cat and Dog.

The training process follows a two-phase strategy. In the first phase, the pretrained backbone is frozen and only the new classification head is trained. This allows the newly initialized classifier head to learn a reasonable mapping from pretrained features to the target labels. In the second phase, the backbone is unfrozen and fine-tuned using a lower learning rate. This allows the network to adjust its internal feature representation to the cat–dog classification task while reducing the risk of large disruptive updates to pretrained weights.

4.3.2 Data Augmentation for Deep Learning

The deep learning pipeline uses stronger dynamic augmentation than the hybrid classical pipeline because the model is trained directly from image batches. During training, images are resized, randomly cropped, randomly flipped, randomly rotated, colour-jittered, converted to tensors, and normalized using ImageNet statistics.

Validation and test images use deterministic transformations only. This is important because evaluation should measure model performance on stable inputs rather than on randomly augmented versions of the same image.

The deep learning augmentation strategy is described in detail in Section 7. In this methodology section, the key point is that stochastic augmentation is used only during training, while validation and testing remain deterministic.

4.3.3 Training Configuration

The deep learning model is trained using AdamW optimization, weight decay, cross-entropy loss with label smoothing, automatic mixed precision, and early stopping based

on validation loss. The completed run uses a batch size of 32 and an input size of 224×224 pixels.

This training setup is more complex than the hybrid classical pipeline because it involves neural network optimization over many parameters, learning-rate control, stochastic augmentation, and checkpoint selection. For this reason, the deep learning pipeline is treated as an extension for comparison rather than the main operational model.

4.4 Configuration-Driven Implementation

The implementation is designed to be **configuration-driven**. Instead of manually modifying code in different notebook sections, major experimental settings are defined through a centralized configuration system. This makes the project easier to reproduce, adjust, and maintain.

The configuration system defines structured settings for paths, runtime options, preprocessing, cleaning, feature extraction, and classifier selection. It also supports classifier schema validation, which reduces configuration errors when switching between models with different hyperparameters. For example, Logistic Regression uses parameters such as regularization strength `C` and maximum iterations, while Random Forest uses parameters such as the number of trees and tree depth.

This design allows alternative configurations to be tested without changing the core pipeline logic. For example, the preprocessing strategy can be changed from stretch resizing to letterbox padding, the backbone can be changed from EfficientNet-B0 to ResNet18 or VGG16, and the classifier can be changed from Logistic Regression to Soft Voting or Stacking.

The project structure also supports reproducibility by separating notebooks, helper modules, generated reports, and saved feature files. The extracted features are stored in `.npy` format, allowing classifier experiments to be rerun without repeating the full image-to-feature extraction process.

4.5 Comparison of Design Decisions

The two modelling approaches differ mainly in how they use pretrained CNN representations. The hybrid classical pipeline uses the CNN as a frozen feature extractor and trains a traditional classifier on saved feature vectors. The deep learning pipeline fine-tunes the CNN directly and adapts the feature representation to the target dataset.

Table 13: Comparison of major design decisions between the two pipelines

Design Aspect	**Hybrid Classical Pipeline**	**Deep Learning Extension**
Input to classifier	Saved pretrained CNN feature vectors	Image tensors processed directly by the neural network
Feature representation	Frozen pretrained representation	Adapted through fine-tuning
Main model	EfficientNet-B0 frozen features + Logistic Regression	Fine-tuned EfficientNet-B0
Training process	Feature extraction followed by traditional classifier training	End-to-end optimization with warm-up and fine-tuning
Computational cost	Lower after feature extraction; features can be reused	Higher because the CNN parameters are optimized during training
Reproducibility	More stable because the backbone is frozen and the classifier is simple	More sensitive to random seed, augmentation, optimizer settings, and checkpoint selection
Main purpose	Required traditional ML pipeline and practical operational model	Bonus extension and comparison with transfer learning

The methodology therefore reflects a deliberate design choice. The hybrid classical pipeline is prioritized because it satisfies the assignment requirement, supports systematic experimentation, and provides a computationally practical workflow. The deep learning extension is included to broaden the comparison and examine whether end-to-end fine-tuning provides additional benefits under a more complex training setup.

5 Experimental Design

This section describes the experimental design used to evaluate the proposed cat–dog image classification system. The purpose of the experiments is not only to obtain a high-performing model, but also to understand how different design choices affect validation performance, computational cost, and reproducibility. These design choices include the cleaning policy, preprocessing strategy, pretrained feature extractor, classifier family, and training paradigm.

The experiments are organized into four main layers. First, a cleaning-threshold experiment is conducted to determine whether quality-based image filtering provides a stable practical benefit. Second, an exhaustive classical grid-search experiment is performed to compare preprocessing strategies, pretrained backbones, and classifier variants. Third, the selected operational configuration is run in the main traditional notebook and evaluated on the held-out test set. Finally, an end-to-end deep learning extension is implemented to compare frozen feature extraction with transfer learning and fine-tuning.

5.1 Experiment Overview

The overall experimental design consists of the following stages:

1. Conduct exploratory data analysis to examine class distribution, image dimensions, RGB colour-channel characteristics, representative samples, and possible data quality issues.
2. Evaluate cleaning thresholds using EfficientNet-B0 as a proxy feature extractor and Logistic Regression as a proxy classifier.
3. Select the final cleaning policy based on performance stability, dataset retention, and class-balance preservation.
4. Apply four preprocessing strategies to the retained dataset: stretch resizing, center cropping, letterbox padding, and augmented preprocessing.
5. Extract feature vectors using three pretrained CNN backbones: ResNet18, VGG16, and EfficientNet-B0.
6. Save extracted feature matrices and labels as NumPy files so that classifier experiments can reuse the same feature representations.
7. Train and evaluate eleven traditional classifier variants on the extracted feature vectors.
8. Compare validation performance across the full classical search space of 132 configurations.
9. Identify the top-ranked grid-search validation candidate.
10. Select the main operational model by considering both predictive performance and computational practicality.
11. Evaluate the selected operational model on the held-out test set.

12. Implement an end-to-end deep learning extension using EfficientNet-B0 transfer learning and fine-tuning.

This design allows the effect of each major pipeline component to be examined systematically. It also avoids selecting the final model purely from a single validation score, because a small validation difference may not justify a substantially higher training and tuning cost.

5.2 Cleaning-Threshold Experiment

Before running the main classification experiments, a separate cleaning-threshold analysis is conducted. The goal is to determine whether filtering potentially weak images, such as blurry, undersized, near-monochrome, or visually extreme samples, improves classification performance in a stable and practically meaningful way.

EfficientNet-B0 is used as a proxy feature extractor to generate feature vectors for all **10,028** images. These extracted features have shape $(10028, 1280)$. A Logistic Regression classifier is then used as a proxy model to compare different cleaning presets. The keep-all baseline already achieves a strong macro F1-score of approximately **0.9935**.

Although some filtering presets produce small improvements under a single random seed, the gains are not stable across seeds and often reduce dataset retention or shift class balance. Therefore, the final selected cleaning policy is a **minimal-safety / keep-all policy**. In the reported traditional experiments, all **10,028** usable images are retained and no images are removed.

Table 14: Cleaning-threshold experiment design

Component	Experimental Setting
Dataset size	10,028 images
Proxy feature extractor	EfficientNet-B0
Proxy feature shape	$(10028, 1280)$
Proxy classifier	Logistic Regression
Baseline policy	Keep all usable images
Baseline proxy macro F1-score	Approximately 0.9935
Selection criteria	Stable macro F1 improvement, high retention, and preserved class balance
Final cleaning policy	Minimal-safety / keep all usable images
Final retention	10,028 / 10,028 images
Removed images	0

This experiment ensures that the cleaning decision is evidence-based rather than purely heuristic. It also clarifies that the final reported pipeline supports image quality checks, but does not aggressively remove blurry, undersized, near-monochrome, or visually difficult samples in the main traditional experiment.

5.3 Classical Grid-Search: Search Space and Protocol

The main classical experiment evaluates a full search space across preprocessing strategies, pretrained backbones, and classifier variants. Unlike a single training run, this exper-

iment is designed to compare multiple controlled configurations under the same validation protocol.

The search space consists of four preprocessing strategies, one input image size, three pretrained backbones, and eleven classifier variants. This produces:

$$4 \times 1 \times 3 \times 11 = 132 \tag{7}$$

classical experiment configurations.

Table 15: Experimental search space for the hybrid classical pipeline

Component	Configurations Tested
Preprocessing strategy	Stretch resize, center crop, letterbox padding, augmented preprocessing
Input image size	224×224 pixels
Pretrained backbone	ResNet18, VGG16, EfficientNet-B0
Classifier variants	Logistic Regression variants, linear SVM variants, Random Forest variants, Soft Voting, and Stacking
Total configurations	132 experiments

All **132/132** configurations are executed successfully in the exhaustive classical experiment. This provides a broader and more reliable basis for model selection than choosing a model from one manually selected pipeline.

5.3.1 Preprocessing Strategies

Four preprocessing strategies are considered in the classical experiment.

The first strategy is **stretch resizing**, which directly resizes each image to 224×224 pixels. This method is simple and computationally efficient, but it may distort the original aspect ratio of the animal.

The second strategy is **center cropping**, in which the image is resized and then cropped around the central region. This method may preserve object details when the animal is located near the center, but it may remove useful visual information when the animal is off-center.

The third strategy is **letterbox padding**, which preserves the original aspect ratio by adding padding around the resized image. This avoids geometric distortion, but the padded regions may introduce artificial borders.

The fourth strategy is **augmented preprocessing**. In the hybrid classical pipeline, this strategy applies lightweight training-time augmentation, including random horizontal flipping and colour jittering, before feature extraction. Validation and testing images are processed using deterministic transforms to ensure stable evaluation.

5.3.2 Backbone Comparison

Three pretrained CNN backbones are evaluated: **ResNet18**, **VGG16**, and **EfficientNet-B0**. These models are used as frozen feature extractors, meaning that their pretrained weights are not updated during the hybrid classical experiments.

ResNet18 is included as a lightweight residual network. VGG16 is included as a deeper sequential convolutional baseline. EfficientNet-B0 is included because it provides an efficient balance between representational strength and computational cost.

For each backbone, the final classification layer is removed. The remaining network produces fixed-dimensional feature vectors, which are saved and reused for classifier training. This design makes classifier comparison more controlled because different classifiers can be trained on the same extracted feature representation.

5.3.3 Classifier Variants

The extracted feature vectors are used as input for traditional machine learning classifiers. The experiment includes eleven classifier variants from the following model families: **Logistic Regression**, **linear Support Vector Machine**, **Random Forest**, **Soft Voting**, and **Stacking**.

Logistic Regression provides a simple and efficient linear decision boundary. It is useful for testing whether the extracted deep features are already linearly separable. Linear SVM is another strong linear classifier for high-dimensional feature spaces. Random Forest is included to test whether a non-linear tree-based ensemble can improve performance on extracted CNN features.

Soft Voting and Stacking are included to evaluate whether combining multiple classifiers improves predictive performance. Soft Voting combines predicted probabilities from base classifiers, while Stacking uses predictions from base models as inputs to a meta-classifier. These ensemble methods may improve validation performance, but they also require more training and tuning time than a single Logistic Regression model.

5.4 Validation and Model Selection Strategy

Model selection is based on both validation performance and computational practicality. During the classical experiment, each candidate configuration is trained using the training split and compared using validation accuracy and macro F1-score. The test set is not used during model selection.

Macro F1-score is used because it evaluates both classes equally. Although the dataset is nearly balanced, macro F1-score remains useful because it shows whether the classifier performs consistently across Cat and Dog classes.

The exhaustive grid-search experiment identifies augmented preprocessing + Efficient Net B0 + Soft Voting as the top-ranked validation candidate, with validation accuracy and macro F1-score of approximately 0.9970. However, the top three configurations have the same validation score at the reported precision. Therefore, Soft Voting is not treated as clearly superior to Logistic Regression. Instead, the final main notebook selects augmented preprocessing + EfficientNet-B0 + Logistic Regression as the main operational configuration because it reaches near-identical validation performance with lower training and tuning cost.

Table 16: Distinction between top-ranked validation candidate and selected operational model

Model Role	Configuration
Top-ranked grid-search validation candidate	Augmented preprocessing + EfficientNet-B0 + Soft Voting
Main operational model	Augmented preprocessing + EfficientNet-B0 + Logistic Regression
Runtime-aware selected configuration	Augmented preprocessing + EfficientNet-B0 + Logistic Regression

This selection strategy avoids over-emphasizing a small or rounded validation-score difference when the computational cost difference is substantial. It also makes the final notebook more suitable for reproducible submission, where the model should be practical to rerun from start to finish.

5.5 Main Operational Configuration

The selected main operational configuration is shown in Table 17. This is the configuration used in the final traditional notebook and reported as the main classical model.

Table 17: Selected main operational configuration for the hybrid classical pipeline

Component	Selected Configuration
Cleaning policy	Minimal-safety / keep all usable images
Preprocessing strategy	Augmented preprocessing
Input resolution	224×224 pixels
Feature extractor	EfficientNet-B0
Feature extraction mode	Frozen pretrained backbone
Feature dimension	1280
Classifier	Logistic Regression
Logistic Regression parameters	`C = 0.1`, `max_iter = 3000`
Split ratio	80% training, 10% validation, 10% testing
Selection basis	Macro F1-score, accuracy, training cost, tuning time, and reproducibility

This configuration is selected because EfficientNet-B0 provides highly discriminative visual features, while Logistic Regression separates these features effectively with low computational cost. The selected model is therefore not chosen as the absolute highest-ranked model in the full grid search; it is chosen as the most practical operational model for a reproducible notebook submission.

5.6 Computational Strategy

The computational strategy is designed to balance experimental coverage and reproducibility. Feature extraction with pretrained CNN backbones is relatively expensive, so

extracted features are saved as NumPy files and reused for classifier training. This avoids repeated CNN forward passes when testing multiple classifiers on the same backbone and preprocessing configuration.

For the main traditional notebook, Logistic Regression is selected because it can be trained and tuned efficiently while maintaining near-top performance. The ensemble configurations, especially Soft Voting and Stacking, require multiple base learners to be fitted for each candidate. Therefore, even when they are competitive in validation performance, their higher training and tuning cost makes them less suitable as the main operational notebook model under practical runtime constraints.

The implementation also follows a reproducible submission structure. The project separates source notebooks, reusable helper modules, report files, saved feature arrays, trained models, and result outputs. In particular, extracted feature vectors are saved in `.npy` format so that later classifier experiments can be rerun without repeating the full feature extraction stage.

5.7 Deep Learning Extension Design

In addition to the required hybrid classical pipeline, an end-to-end deep learning extension is implemented using EfficientNet-B0 transfer learning and fine-tuning. This extension is included to compare the traditional feature-extraction approach with a neural network trained directly from image inputs.

The deep learning pipeline uses a two-phase training strategy. In the first phase, the pretrained backbone is frozen and only the classification head is trained. In the second phase, the backbone is unfrozen and fine-tuned with a lower learning rate. This allows the model to adapt its representation to the cat–dog classification task while reducing the risk of large disruptive updates to pretrained weights.

The deep learning extension uses the same original dataset and a similar evaluation protocol, but it applies an additional spatial outlier handling step. Therefore, its final test size differs slightly from the hybrid classical pipeline. For this reason, the comparison between the two pipelines is treated as indicative rather than perfectly one-to-one.

5.8 Summary

The experimental design evaluates the project from four perspectives: cleaning-policy selection, exhaustive classical configuration search, main operational model selection, and deep learning extension. The classical search space contains **132 configurations** formed by four preprocessing strategies, one input size, three pretrained backbones, and eleven classifier variants.

The key design decision is to separate the **top-ranked grid-search validation candidate** from the **main operational model**. Soft Voting is ranked first in the exhaustive validation table, but Logistic Regression achieves the same rounded validation score with substantially lower training and tuning cost. Therefore, Logistic Regression is selected for the final traditional notebook because it provides a more practical balance between performance, computational efficiency, simplicity, and reproducibility. The following section presents the hybrid classical pipeline results in detail.

6 Results: Hybrid Classical Pipeline

This section presents the experimental results of the **hybrid classical feature-extraction pipeline**. The discussion focuses on the exhaustive classical grid-search results, the effect of pretrained backbone selection, the effect of preprocessing strategy, classifier-level differences, and the final test performance of the selected main operational model.

A key distinction is maintained throughout this section. The **top-ranked validation candidate** in the exhaustive grid-search experiment is the combination of augmented preprocessing, EfficientNet-B0 feature extraction, and Soft Voting. However, the **main operational model** reported in the final traditional notebook is augmented preprocessing, EfficientNet-B0 frozen feature extraction, and Logistic Regression. This decision is made because Logistic Regression reaches the same rounded validation score as the top-ranked candidate while requiring substantially lower training and tuning cost.

6.1 Grid-Search Results Overview

An exhaustive classical grid-search experiment was conducted to compare different combinations of preprocessing strategies, pretrained backbones, and classifier variants. The experiment covered four preprocessing strategies, one input image size, three pretrained backbones, and eleven classifier variants, producing a total of:

$$4 \times 1 \times 3 \times 11 = 132 \tag{8}$$

classical experiment configurations. All **132/132** experiments were completed successfully, with no failed runs.

The top validation configurations are summarized in Table 18. The results show that EfficientNet-B0 dominates the highest-ranked configurations, especially when combined with augmented preprocessing.

Table 18: Top-performing configurations from the exhaustive classical grid-search experiment

Rank	Preprocessing	Backbone	Classifier Variant	Validation Score
1	Augmented_224	EfficientNet-B0	Voting Soft	0.997009
2	Augmented_224	EfficientNet-B0	Logistic Regression	0.997009
3	Augmented_224	EfficientNet-B0	Stacking	0.997009
4	Letterbox_224	EfficientNet-B0	Stacking	0.996012
5	Stretch_224	EfficientNet-B0	Stacking	0.995015
6	CenterCrop_224	EfficientNet-B0	Logistic Regression	0.995015
7	Letterbox_224	EfficientNet-B0	Voting Soft	0.995015
8	Letterbox_224	EfficientNet-B0	Logistic Regression	0.995015
9	Augmented_224	EfficientNet-B0	Logistic Regression	0.995015
10	CenterCrop_224	EfficientNet-B0	LinearSVM	0.995015

Table 18 shows that the three best validation candidates all use **augmented preprocessing** and **EfficientNet-B0** features. Soft Voting is ranked first, but Logistic Regression and Stacking obtain the same validation score at the reported precision. Therefore,

the result should not be interpreted as evidence that Soft Voting clearly outperforms Logistic Regression. Instead, the table indicates that the strongest feature representation comes from EfficientNet-B0 under the augmented preprocessing setting, while several classifiers can perform almost identically on that representation.

This result is important for model selection. Although Soft Voting is the top-ranked validation candidate, Logistic Regression reaches the same rounded validation score with a much simpler training and tuning process. For this reason, Logistic Regression is selected as the main operational model, while Soft Voting is reported as the strongest ensemble candidate from the exhaustive validation search.

6.2 Effect of the Pretrained Backbone

The pretrained backbone is one of the most influential components of the hybrid classical pipeline. Among the three tested backbones, **EfficientNet-B0** provides the strongest overall feature representation and appears consistently in the highest-ranked configurations.

This result is reasonable because EfficientNet-B0 is designed to balance network depth, width, and input resolution efficiently. For the cat–dog classification task, its extracted features appear to capture discriminative visual cues such as object shape, texture, facial structure, fur pattern, and body characteristics. Once these high-level features are extracted, the downstream classifier no longer needs to learn directly from raw pixels.

In comparison, VGG16 remains a useful convolutional baseline, but it does not dominate the top-ranked results. ResNet18 is computationally lightweight and useful for comparison, but its extracted features are generally less competitive than those produced by EfficientNet-B0 in this experiment.

The backbone comparison suggests that stronger pretrained visual representations make the downstream classification task easier. When the feature extractor already separates the two classes well, even a simple classifier can achieve strong performance.

6.3 Effect of Preprocessing Strategy

The preprocessing strategy also affects validation performance. Among the tested methods, **augmented preprocessing** gives the best overall result when combined with EfficientNet-B0.

Stretch resizing is simple and efficient, but it may distort the original aspect ratio of the animal. Center cropping can preserve central object details, but it may remove useful visual information when the animal is not located near the image center. Letterbox padding preserves the original aspect ratio, but the added padding may introduce artificial border regions that are not part of the animal object.

Augmented preprocessing provides a better practical balance because it standardizes image size while introducing controlled variation during training. In the hybrid classical pipeline, this strategy mainly uses random horizontal flipping and colour jittering before frozen feature extraction. These transformations improve robustness to natural changes in animal orientation, lighting condition, contrast, and colour tone.

6.4 Effect of Classifier Choice

The classifier comparison shows that several traditional classifiers perform very well when trained on EfficientNet-B0 features. This suggests that the extracted feature space is already highly discriminative for the Cat and Dog classes.

Soft Voting and Stacking appear among the top validation configurations, showing that ensemble-based methods can be competitive. Soft Voting combines predicted probabilities from multiple base classifiers, while Stacking trains an additional meta-classifier on top of base-model outputs. These methods can improve validation performance in some settings because they combine different decision patterns.

However, the gain from ensemble methods is marginal in this experiment. The top-ranked Soft Voting model and the best Logistic Regression variant obtain the same validation score at the reported precision. In practice, the ensemble configurations require substantially longer training and tuning time because multiple base learners are fitted for each candidate. Stacking is especially expensive because it trains base classifiers and an additional meta-classifier.

By contrast, **Logistic Regression** is much easier to train and tune. It has fewer moving parts, simpler hyperparameters, and more stable rerun behavior. With EfficientNet-B0 features, Logistic Regression remains near-identical to the top ensemble candidate on validation performance and achieves strong final test results. Therefore, Logistic Regression is selected as the **main operational classifier**, while Soft Voting is reported as the **top-ranked ensemble candidate** from the grid search.

6.5 Selected Main Operational Model

Based on the full experimental process, the selected main operational model uses **augmented preprocessing**, **EfficientNet-B0 frozen feature extraction**, and **Logistic Regression**. This configuration is used in the final traditional notebook because it provides the most practical balance between predictive performance, runtime cost, simplicity, and reproducibility.

The Logistic Regression model is tuned using GridSearchCV with three-fold cross-validation on the training features. The tuning process evaluates regularization strength and maximum iterations using macro F1-score as the refit metric. The best parameters are `C = 0.1` and `max_iter = 3000`.

On the validation set, the final Logistic Regression model achieves approximately **0.9960** accuracy and **0.9960** macro F1-score, with **4** wrong predictions out of **1,003** validation images. This confirms that the selected operational model remains highly competitive despite being simpler than ensemble alternatives.

6.6 Final Test Performance

After the main operational configuration is selected, the final Logistic Regression model is evaluated on the held-out test set. The test set is not used during model selection, making it suitable for estimating final model performance.

Table 19: Final test performance of the selected Logistic Regression model

Metric	Value
Test accuracy	0.9940
Macro F1-score	0.9940
Number of test images	1,003
Correct predictions	997
Incorrect predictions	6
Error rate	0.60%

The final selected operational model achieves approximately **0.9940** test accuracy and **0.9940** macro F1-score. Out of **1,003** test images, **997** are classified correctly and **6** are misclassified.

The close accuracy and macro F1-score indicate that the model performs evenly across both classes, which is consistent with the balanced test distribution. The result also confirms that EfficientNet-B0 frozen features are highly discriminative for this binary classification task.

Because the score is very high, the result should still be interpreted within the current dataset and split. The experiment shows strong generalization to the held-out test set, but it does not guarantee the same performance on external datasets with different image sources, resolutions, lighting conditions, backgrounds, or label quality.

6.7 Class-Level Performance and Confusion Matrix

Table 20 presents the class-level classification results for the final selected Logistic Regression model.

Table 20: Class-level performance of the final selected Logistic Regression model

Class	Precision	Recall	F1-score	Support
Cat	0.9920	0.9960	0.9940	501
Dog	0.9960	0.9920	0.9940	502
Macro Average	0.9940	0.9940	0.9940	1,003
Weighted Average	0.9940	0.9940	0.9940	1,003

Both classes achieve very high precision, recall, and F1-score. The Cat class obtains slightly higher recall, while the Dog class obtains slightly higher precision. However, the difference is very small, and the macro average remains approximately 0.9940. This indicates that the classifier does not strongly favor either class.

Table 21 provides a more detailed view of the model's prediction behavior on the test set.

Table 21: Confusion matrix of the final selected Logistic Regression model on the test set

	Predicted Cat	Predicted Dog
Actual Cat	499	2
Actual Dog	4	498

The confusion matrix shows that the model makes **6** errors on the test set. Specifically, **2** Cat images are predicted as Dog, while **4** Dog images are predicted as Cat. Although the error direction is slightly higher for Dog images, the difference is small relative to the full test set. Therefore, the model does not show a strong class bias.

The small number of misclassifications suggests that EfficientNet-B0 provides a highly effective feature representation for this dataset. Since Logistic Regression can separate the extracted features effectively, the feature space appears close to linearly separable for the cat–dog classification task.

6.8 Misclassified Sample Analysis

Although the model achieves strong performance, analyzing the misclassified samples is still useful. Figure 5 presents examples of images that were incorrectly classified by the final selected Logistic Regression model.

Figure 5: Misclassified samples from the final selected Logistic Regression model. Each image is shown with its true label and predicted label.

The **six** misclassified images are useful for checking the remaining failure cases of the final model. These errors may be related to unusual pose, partial visibility, background clutter, lighting conditions, low resolution, or visual similarity between the two classes. Since only six errors remain out of 1,003 test images, the qualitative inspection does not indicate a systematic failure pattern.

This analysis also helps verify that the errors are not caused by an obvious implementation issue. Instead, they appear to be difficult visual cases that can naturally occur in real-world image classification.

6.9 Discussion and Summary of Findings

The experimental results show that the quality of the extracted visual representation is the most important factor in the hybrid classical pipeline. EfficientNet-B0 gives the strongest result among the tested backbones and appears consistently among the top-ranked configurations. This confirms that a strong pretrained feature extractor can make the downstream classification task much easier.

The results also explain why Logistic Regression performs strongly. After EfficientNet-B0 feature extraction, the classification problem is no longer based on raw pixels. Instead, the classifier receives compact 1280-dimensional feature vectors that already contain high-level visual information. If these features separate Cat and Dog images clearly, a simple linear classifier can perform very well.

Ensemble methods remain valuable, but their practical advantage is limited in this experiment. Soft Voting is the top-ranked validation candidate, yet Logistic Regression

obtains the same rounded validation score and is much cheaper to train and tune. Therefore, the final model selection follows a runtime-aware criterion rather than a purely score-based criterion.

The main findings from the hybrid classical pipeline experiments are summarized as follows:

- The exhaustive classical grid-search experiment covers **132 configurations**, and all experiments are completed successfully.
- **EfficientNet-B0** provides the strongest feature representation among the tested backbones.
- **Augmented preprocessing** achieves the best overall validation performance among the tested image transformation strategies.
- **Soft Voting** is the top-ranked validation candidate, but the top three configurations have the same validation score at the reported precision.
- **Logistic Regression** is selected as the main operational classifier because it reaches near-identical validation performance at lower training and tuning cost.
- The final selected Logistic Regression model achieves approximately **0.9940** test accuracy and **0.9940** macro F1-score on the held-out test set.
- The final selected model misclassifies **6** images out of **1,003** test images, corresponding to an error rate of approximately **0.60%**.
- The confusion matrix and class-level metrics show balanced performance across both Cat and Dog classes.
- The hybrid classical pipeline is computationally practical because feature extraction and classifier training are separated, and extracted features can be reused across classifier experiments.

Overall, the results confirm that combining pretrained deep feature extraction with a traditional machine learning classifier is an effective strategy for binary image classification. The experiment also shows that the highest validation rank is not the only criterion for final model selection. In this project, Logistic Regression is preferred as the main operational model because it maintains strong performance while being easier to reproduce than ensemble alternatives.

The following section presents the end-to-end deep learning extension, which is implemented as an additional experiment to compare transfer learning and fine-tuning against the hybrid classical feature-extraction pipeline.

7 Results: End-to-End Deep Learning Extension

This section presents the results of the end-to-end deep learning extension. While the main required pipeline of this project is the hybrid classical machine learning pipeline, the deep learning extension is implemented to examine whether transfer learning and fine-tuning can provide additional benefits when the pretrained backbone is adapted directly to the cat–dog classification task.

The deep learning model uses EfficientNet-B0 as the pretrained backbone. As described in Section 4, the model is trained using a two-phase transfer learning strategy. In the first phase, only the classification head is trained while the backbone remains frozen. In the second phase, the full network is unfrozen and fine-tuned using a lower learning rate. This training strategy is designed to stabilize convergence while still allowing the model to adapt its internal representation to the target dataset.

The deep learning pipeline uses the same original Cat–Dog dataset as the hybrid classical pipeline and follows a similar train-validation-test evaluation protocol. However, it applies an additional spatial outlier handling step, which leads to a slightly different final test size. Therefore, the results in this section should be interpreted as a strong extension experiment, while the direct comparison with the hybrid classical pipeline remains indicative rather than strictly split-identical.

7.1 Training Dynamics

The completed deep learning run shows a clear two-stage training pattern corresponding to the head warm-up phase and the full fine-tuning phase.

During the first three epochs, only the classification head is trained. Training accuracy increases from 82.45% to 91.66%, while validation accuracy reaches 94.90%. Validation loss decreases from 0.4061 to 0.3404. This indicates that the newly initialized classification head learns to map the frozen EfficientNet-B0 representations to the two target classes.

After the backbone is unfrozen at epoch 4, validation performance improves sharply. Validation accuracy increases from 94.90% at epoch 3 to 97.70% at epoch 4, while validation loss decreases from 0.3404 to 0.2673. This suggests that full fine-tuning helps the pretrained backbone adapt its representation to the cat–dog classification task.

The best validation loss is reached at epoch 14, where the model obtains 99.20% validation accuracy and a validation loss of 0.2273. Training continues until epoch 19, where early stopping is triggered after no further validation-loss improvement within the patience window. The best checkpoint from epoch 14 is therefore selected for final test evaluation.

Table 22: Selected epoch-level training history from the completed deep learning run

Epoch	Phase	Train Loss	Train Acc	Val Loss	Val Acc
1	Warm-up	0.5221	82.45%	0.4061	93.90%
2	Warm-up	0.4019	90.60%	0.3521	94.60%
3	Warm-up	0.3739	91.66%	0.3404	94.90%
4	Fine-tuning	0.3269	93.98%	0.2673	97.70%
7	Fine-tuning	0.2640	97.00%	0.2381	98.60%
11	Fine-tuning	0.2505	97.89%	0.2307	98.90%
14	Fine-tuning	0.2420	98.40%	**0.2273**	**99.20%**
19	Fine-tuning	0.2404	98.41%	0.2297	98.70%

Best checkpoint saved at epoch 14 with validation loss = 0.2273.

Overall, the training dynamics show that the two-phase strategy is effective. The warm-up phase allows the classification head to learn a stable initial mapping, while the fine-tuning phase improves validation performance by adapting the pretrained backbone to the target task.

7.2 Final Test Performance

The best checkpoint from epoch 14 is loaded and evaluated on the held-out test set. The model achieves **0.9870** test accuracy and **0.9870** macro F1-score, with **13** misclassified images out of **1,001** test samples. The ROC-AUC score reaches **0.9992**, indicating that the model's predicted scores are highly separable between the two classes.

Table 23: Final test performance of the deep learning pipeline

Metric	Value
Test accuracy	0.9870
Macro F1-score	0.9870
ROC-AUC	0.9992
Total test images	1,001
Correct predictions	988
Incorrect predictions	13
Error rate	1.30%

The final result shows that the deep learning extension performs strongly on the cat–dog classification task. Although the test accuracy is slightly lower than the selected hybrid classical model, the ROC-AUC value suggests that the model still separates the two classes very well in terms of predicted confidence scores.

7.3 Class-Level Performance

Table 24 presents the class-level results of the deep learning model on its held-out test set.

Table 24: Class-level performance of the deep learning model on the test set

Class	Precision	Recall	F1-score	Support
Cat	0.9900	0.9840	0.9870	500
Dog	0.9840	0.9900	0.9870	501
Macro Average	0.9870	0.9870	0.9870	1,001
Weighted Average	0.9870	0.9870	0.9870	1,001

Both classes achieve nearly identical F1-scores, indicating that the model does not strongly favor either class. Cat images obtain slightly higher precision, while Dog images obtain slightly higher recall. The macro and weighted averages are both approximately 0.9870, confirming balanced performance across the two categories.

This class-level result is important because the dataset is nearly balanced. If the model were biased toward one class, this would appear as a noticeable difference in recall or F1-score. Instead, the results show that the deep learning model maintains consistent performance on both Cat and Dog images.

7.4 Discussion

The deep learning extension confirms that EfficientNet-B0 is a strong backbone for the cat–dog classification task. The model reaches high validation accuracy during fine-tuning and achieves strong final test performance. The two-phase training process also behaves as expected: the head warm-up phase provides a stable starting point, while the fine-tuning phase improves the model by adapting the pretrained representation.

The result also shows that end-to-end fine-tuning is effective but more complex than the hybrid classical approach. The deep learning pipeline requires stochastic augmentation, optimizer tuning, learning-rate scheduling, mixed precision, early stopping, and checkpoint selection. These components are useful for improving generalization, but they also make the final result more sensitive to training dynamics and random seed.

The completed run achieves **0.9870** test accuracy, **0.9870** macro F1-score, and **0.9992** ROC-AUC. These results indicate strong performance, but they should not be interpreted as the definitive best possible result of the deep learning approach. Only one completed fine-tuning run is reported, so repeated runs across multiple random seeds would make the conclusion stronger.

The slightly different test size should also be considered when interpreting the result. The deep learning pipeline uses an additional spatial outlier handling step, resulting in **1,001** test images instead of the **1,003** images used in the traditional pipeline. Therefore, the comparison with the hybrid classical model should be discussed as an indicative comparison rather than a perfectly one-to-one evaluation.

Overall, the deep learning extension provides useful evidence that transfer learning and fine-tuning can solve the cat–dog classification task effectively. However, within the current experimental setup, its main value is as a strong comparative extension rather than as the selected operational pipeline. The next section compares the hybrid classical pipeline and the deep learning extension in terms of performance, computational cost, reproducibility, and practical suitability.

8 Comparative Analysis

This section compares the main hybrid classical pipeline with the end-to-end deep learning extension. The purpose of this comparison is not only to identify which approach obtains the higher test score, but also to examine the trade-offs between predictive performance, computational cost, implementation complexity, reproducibility, and practical suitability.

The comparison uses the selected main operational model from the hybrid classical pipeline: **EfficientNet-B0 frozen feature extraction followed by Logistic Regression**. Although Soft Voting is the top-ranked validation candidate in the exhaustive classical grid-search experiment, Logistic Regression is used in this comparison because it is the selected operational model in the final traditional notebook.

The deep learning extension uses **EfficientNet-B0 transfer learning and fine-tuning**. Both pipelines are based on the same original Cat–Dog dataset and follow a similar train-validation-test evaluation protocol. However, the deep learning pipeline applies an additional spatial outlier handling step, resulting in a slightly different final test size. Therefore, the comparison should be interpreted as **indicative** rather than strictly split-identical.

8.1 Performance Comparison

Table 25 summarizes the main performance and design differences between the two pipelines.

Table 25: Performance comparison between the hybrid classical pipeline and the deep learning extension

Criterion	Hybrid Classical Pipeline	Deep Learning Extension
Main model	EfficientNet-B0 frozen features + Logistic Regression	EfficientNet-B0 transfer learning and fine-tuning
Feature representation	Pretrained CNN features saved as 1280-dimensional vectors	CNN representation adapted through end-to-end fine-tuning
Test accuracy	0.9940	0.9870
Macro F1-score	0.9940	0.9870
Incorrect predictions	6 / 1,003	13 / 1,001
ROC-AUC	Not reported	0.9992
Training paradigm	Frozen feature extraction followed by traditional classifier training	Two-phase neural network training with head warm-up and full fine-tuning
Model selection	Logistic Regression selected for performance, efficiency, and reproducibility	Best checkpoint selected from the completed fine-tuning run
Test-set note	Traditional test split contains 1,003 images	Deep learning test split contains 1,001 images due to additional spatial outlier handling

The hybrid classical pipeline achieves the higher reported test accuracy and macro F1-score in this experiment, with **0.9940** test accuracy and **0.9940** macro F1-score. It misclassifies only **6** images out of **1,003** test samples. The deep learning extension also performs strongly, achieving **0.9870** test accuracy and **0.9870** macro F1-score, with **13** misclassified images out of **1,001** test samples.

This result suggests that the pretrained EfficientNet-B0 representation is already highly discriminative for the cat–dog classification task. When these features are extracted and passed to Logistic Regression, the classifier can separate the two classes effectively using a simple linear decision boundary. Therefore, full fine-tuning does not provide a clear performance advantage under the current experimental setup.

However, the deep learning extension should not be considered weak. Its **0.9992 ROC-AUC** indicates that the model's predicted scores are highly separable between Cat and Dog images. The slightly lower accuracy and macro F1-score may be affected by training dynamics, stochastic augmentation, checkpoint selection, random seed, or the slightly different data preparation protocol.

8.2 Computational Cost and Reproducibility

The two pipelines differ substantially in computational cost and reproducibility.

The hybrid classical pipeline requires an initial feature extraction stage, where images are passed through a pretrained CNN backbone. This stage can be computationally expensive, but it only needs to be performed once for each preprocessing-backbone configuration. After the features are saved as `.npy` files, classical classifiers can be trained, tuned, and compared much more efficiently.

Logistic Regression is especially practical in this setting. It has relatively few hyperparameters, trains quickly on fixed feature vectors, and is easier to rerun than ensemble methods or end-to-end neural networks. This makes the final notebook more stable and suitable for reproducible submission.

The deep learning extension is more computationally demanding. It requires multiple epochs of neural network optimization, GPU acceleration for practical training time, learning-rate scheduling, stochastic augmentation, mixed precision, early stopping, and checkpoint management. These components are useful for improving model performance, but they also make the final result more sensitive to random seed, augmentation behavior, optimizer settings, and fine-tuning decisions.

Table 26: Computational and reproducibility comparison

Aspect	Hybrid Classical Pipeline	Deep Learning Extension
Main cost	CNN feature extraction, performed once per configuration	Repeated forward and backward passes during fine-tuning
Classifier training	Fast after features are extracted	Integrated into full neural network training
GPU requirement	Useful for feature extraction, but classifier training is lightweight	Practically required for efficient fine-tuning
Reusability	Saved feature files can be reused across classifiers	Full model training must be repeated for new training settings
Reproducibility	More stable because the backbone is frozen and the final classifier is simple	More sensitive to random seed, augmentation, optimizer settings, and checkpoint selection
Runtime suitability	Better suited for a notebook that must be rerun from start to finish	Better suited when computational resources and tuning time are available

From a computational perspective, the hybrid classical pipeline is more practical for this project. It separates expensive feature extraction from classifier training, allowing the same feature vectors to be reused across multiple experiments. This is especially important because the classical grid search covers **132 configurations**. Without feature storage, repeatedly extracting CNN features for each classifier comparison would be inefficient.

8.3 Practical Suitability

The hybrid classical pipeline is more suitable when the priority is a strong, fast, and reproducible workflow. It satisfies the required traditional machine learning pipeline structure while still benefiting from pretrained deep visual representations. The final Logistic Regression model is simple, efficient, and achieves very high test performance.

The deep learning extension is more suitable when task-specific feature adaptation is important. Fine-tuning allows the pretrained backbone to adjust its internal representation to the target dataset, which may become more valuable for datasets that are more different from ImageNet, contain more classes, or require more fine-grained visual distinctions. However, this flexibility comes with higher training complexity and greater sensitivity to hyperparameters.

In this project, the hybrid classical pipeline is selected as the main operational workflow. This does not mean that the deep learning extension is ineffective. Instead, it shows that for this dataset, frozen EfficientNet-B0 features are already strong enough for a simple traditional classifier to perform extremely well. The additional complexity of full fine-tuning does not produce a higher test score in the completed run.

8.4 Key Takeaways

The comparison between the two pipelines leads to several key observations:

- Both approaches perform strongly on the cat–dog classification task, confirming that EfficientNet-B0 provides highly useful visual representations.
- The hybrid classical pipeline achieves the higher reported test accuracy and macro F1-score in this experiment.
- Logistic Regression performs very well because EfficientNet-B0 already extracts discriminative 1280-dimensional feature vectors.
- Soft Voting is the top-ranked validation candidate in the classical grid search, but Logistic Regression is selected as the main operational model because it reaches the same rounded validation score with lower training and tuning cost.
- The deep learning extension achieves strong results and a very high ROC-AUC score, but it requires more complex optimization and is more sensitive to training dynamics.
- The comparison is indicative rather than perfectly split-identical because the deep learning pipeline applies an additional spatial outlier handling step.
- For the final submitted workflow, the hybrid classical pipeline is the more practical choice because it provides strong performance, efficient reruns, saved features, and better reproducibility.

Overall, the comparative analysis supports the main conclusion of the project: pre-trained deep visual features are highly effective for binary cat–dog classification, and a traditional classifier can perform extremely well when the feature representation is strong. The deep learning pipeline remains valuable as a bonus extension and as evidence that transfer learning can also solve the task effectively, but the hybrid classical pipeline offers the better practical balance for the final reported system.

9 Limitations and Future Work

Although the proposed system achieves strong performance on the cat–dog classification task, several limitations should be considered when interpreting the results. This section discusses the main limitations of the current study and suggests possible directions for future improvement.

9.1 Limitations

The first limitation is the scope of the task. This project focuses on a **binary classification** problem with only two classes: *Cat* and *Dog*. While this task is useful for evaluating a complete image classification pipeline, it is less complex than multi-class or fine-grained recognition problems. In real-world applications, visual recognition systems often need to distinguish between many categories with more subtle differences. Therefore, the high performance reported in this study should be interpreted within the scope of binary cat–dog classification.

The second limitation is related to **dataset dependency**. The final models are evaluated using held-out test sets derived from the same original Kaggle dataset. Although the test sets are separated from the training and validation sets, they still come from the same overall data source. As a result, the reported performance reflects generalization within the selected dataset, but it does not fully guarantee robustness on external real-world images. Images from other sources may differ in camera quality, resolution, background complexity, lighting condition, animal breed, or label quality.

The third limitation concerns possible **duplicate or near-duplicate images**. If visually similar images appear across training and testing subsets, the evaluation result may become overly optimistic. The current pipeline supports duplicate-related checks, but the final reported cleaning policy retains all usable images because the cleaning-threshold experiment does not show a stable practical benefit from aggressive filtering. A more rigorous duplicate-removal procedure before splitting would make the evaluation protocol stronger.

The fourth limitation is related to the **minimal-safety cleaning policy**. The cleaning-threshold experiment empirically justifies keeping all 10,028 usable images because aggressive filtering does not provide a stable improvement while preserving retention and class balance. However, this also means that some difficult or low-quality images may remain in the dataset. These samples may still affect model training or evaluation, especially if the task is extended to a more complex dataset.

The fifth limitation is that the hybrid classical pipeline relies heavily on **ImageNet-pretrained representations**. This is generally beneficial because ImageNet-pretrained models contain strong visual features for natural images. However, if the target dataset were substantially different from ImageNet, frozen pretrained features might become less effective, and end-to-end fine-tuning could become more important.

The sixth limitation concerns the **model-selection decision**. In the exhaustive classical grid-search experiment, Soft Voting is the top-ranked validation candidate. However, Logistic Regression achieves the same rounded validation score while requiring substantially lower training and tuning cost. Therefore, Logistic Regression is selected as the main operational model. This is a reasonable runtime-aware decision for a reproducible notebook submission, but it also means that the final reported workflow prioritizes practical reproducibility over using the most complex ensemble candidate.

The seventh limitation is the **non-identical comparison** between the hybrid classical pipeline and the deep learning extension. The two pipelines use the same original dataset and follow a similar evaluation protocol, but the deep learning pipeline applies an additional spatial outlier handling step. As a result, its final test size is 1,001 images, while the traditional pipeline uses 1,003 test images. Therefore, the comparison should be interpreted as indicative rather than perfectly one-to-one.

The final limitation concerns the **deep learning extension**. Only one completed fine-tuning run is reported. Since neural network training is sensitive to random seed, learning rate, batch size, augmentation strength, regularization, optimizer settings, and checkpoint selection, the reported deep learning result may not represent the best possible performance of the end-to-end approach. Repeated runs across multiple seeds would make the conclusion stronger.

9.2 Future Work

Several improvements can be made in future work. First, the model should be evaluated on an **external cat–dog dataset** collected from a different source. This would provide a stronger test of real-world generalization and help determine whether the reported performance remains stable beyond the current Kaggle dataset.

Second, duplicate and near-duplicate detection should be applied **before dataset splitting**. Techniques such as perceptual hashing, image similarity search, or feature-space clustering can be used to identify visually similar images. Removing near-duplicates before splitting would reduce the possibility of overly optimistic evaluation results.

Third, future experiments should compare Logistic Regression and Soft Voting under a clearer **fixed computational budget**. Since Soft Voting is the top-ranked validation candidate while Logistic Regression achieves the same rounded validation score at lower cost, a controlled runtime and performance comparison would help quantify whether the ensemble provides enough practical benefit to justify its additional complexity.

Fourth, the traditional and deep learning pipelines should be evaluated using a **fully shared fixed split** whenever possible. This would make the comparison more direct by ensuring that both pipelines are tested on exactly the same images. If one pipeline requires additional outlier handling, that difference should be reported separately and evaluated as part of the experimental protocol.

Fifth, additional pretrained backbones could be explored. In addition to ResNet18, VGG16, and EfficientNet-B0, future experiments could include more recent architectures such as ConvNeXt, EfficientNetV2, Vision Transformer, or Swin Transformer. These models may provide stronger feature representations or different computational trade-offs.

Sixth, model interpretability methods could be included. Techniques such as Grad-CAM can visualize which image regions influence the model's prediction. This would make the classification decision more transparent and help verify whether the model focuses on meaningful animal features rather than irrelevant background patterns.

Seventh, the deep learning extension could be improved through more systematic hyperparameter tuning and repeated training runs. Possible improvements include testing different fine-tuning depths, adjusting learning rates, increasing the number of epochs, changing augmentation strength, applying stronger regularization, and repeating experiments across multiple random seeds.

Finally, the project could be extended from binary classification to **multi-class or fine-grained animal classification**. For example, the model could be trained to classify

different animal species or different cat and dog breeds. This would make the task more challenging and provide a more comprehensive evaluation of the pipeline.

9.3 Summary

The main limitations of this study arise from the binary nature of the task, dataset-specific evaluation, possible near-duplicate samples, the keep-all cleaning policy, the runtime-aware model-selection decision, the non-identical comparison between the traditional and deep learning pipelines, and the single completed deep learning run.

Despite these limitations, the project successfully demonstrates that pretrained deep feature extraction combined with traditional machine learning classifiers can achieve strong and efficient performance on image classification. Future work should focus on external validation, stricter duplicate handling before splitting, controlled comparison between Logistic Regression and Soft Voting, shared fixed splits for fairer pipeline comparison, repeated deep learning runs, model interpretability, and extension to more complex classification tasks.